

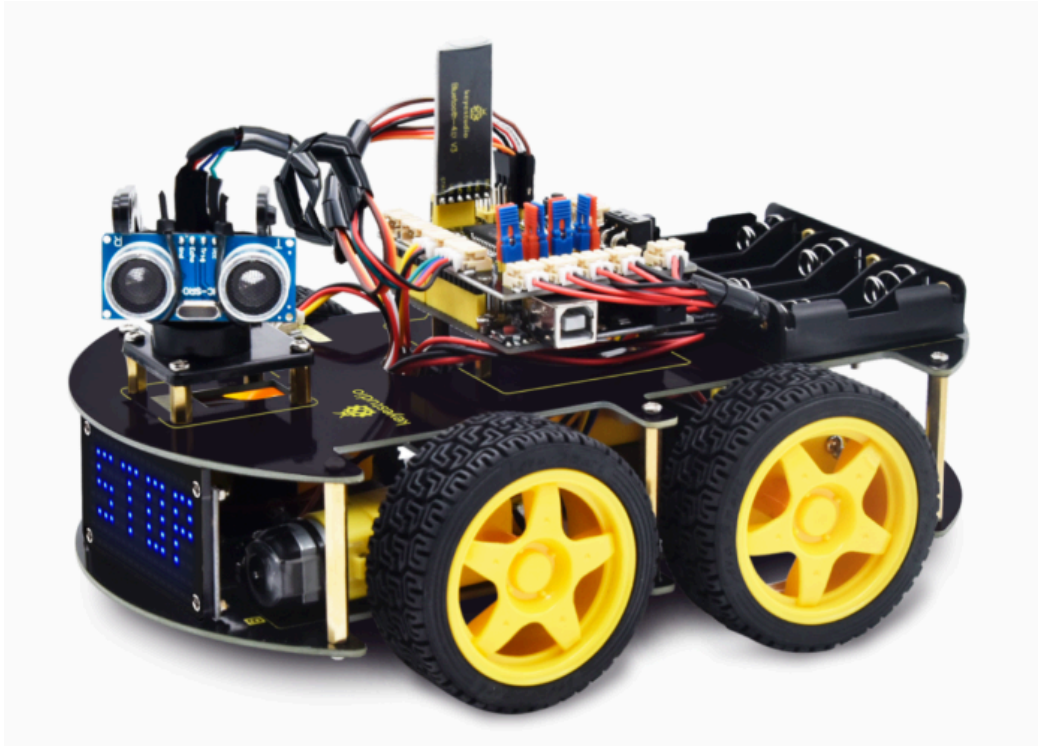
4WD BT Multi-purpose Car V2.0 Kit Arduino Tutorial

1.1 Introducción

La sociedad moderna está siendo impulsada a un ritmo asombroso por los avances tecnológicos, y la educación STEM (educación integrada en Ciencia, Tecnología, Ingeniería y Matemáticas) está recibiendo una atención creciente como una forma clave de cultivar la próxima generación de talentos innovadores.

Arduino, como una herramienta que une el estudio de la programación, la operación de hardware y la realización creativa, es sin duda una plataforma ideal para la educación STEM. No solo sirve como un entorno de aprendizaje de programación y hardware, sino que también inspira la creatividad de los estudiantes y despierta el entusiasmo por el aprendizaje.

El coche Keystudio 4WD ofrece hasta 15 proyectos de aprendizaje, con dificultades que progresan paso a paso. Estos van desde aplicaciones simples de sensores hasta la construcción de un complejo robot inteligente de tracción en las cuatro ruedas, ayudando a los estudiantes a comprender sistemáticamente los principios y las aplicaciones del mundo real de los sensores y módulos de hardware.



1.2 Características

- **Función multiusos:** Evasión de obstáculos, seguimiento, control remoto IR, control Bluetooth, seguimiento ultrasónico y visualización de emoticonos faciales.
- **Fácil de construir:** No se requiere soldadura de circuitos, montaje completo sencillo.
- **Alta tenacidad:** Soporte de aleación de aluminio, motores metálicos, ruedas y orugas de alta calidad.
- **Alta extensión:** Amplíe otros sensores y módulos a través del protector del controlador del motor y el protector del sensor.
- **Controles múltiples:** Control remoto IR, control por App (sistemas iOS y Android).
- **Programación básica:** Código en lenguaje C de Arduino IDE.

1.3 Especificaciones

Disipación de potencia máxima: 25W (T=75°C)

Voltaje de trabajo	5V
--------------------	----

Voltaje de entrada	7-12V
Corriente de salida máxima	2A
Velocidad del motor	5V 200 rpm/min
Modo de accionamiento del motor	Accionamiento por puente H doble
Ángulo de inducción ultrasónica	< 15 grados
Distancia de detección ultrasónica	2cm - 400cm
Distancia de control remoto infrarrojo	10 metros (medido)
Distancia de control remoto Bluetooth	50 metros (medido)
Control Bluetooth	Soporta sistemas Android e iOS

1.4 Lista de Productos

No.	Nombre	Cant.
1	Placa Keystudio V4.0	1
2	Escudo Driver de Motor Keystudio	1
3	Keystudio HM-10 Bluetooth-4.0	1
4	Módulo LED Rojo	1
5	Sensor Ultrasónico HC-SR04	1
6	Sensor de Seguimiento de Línea Keystudio	1

No.	Nombre	Cant.
7	Sensor Receptor IR Keystudio	1
8	Matriz de Puntos LED 8*16 Keystudio (con línea Dupont de 4 pines)	1
9	Servo 9G Keystudio	1
10	Control Remoto Keystudio	1

11	Cable USB	1
12	Soporte para Batería 18650	1
13	Soporte para 6 Baterías AA	1
14	Plataforma de Servo	1
15	Cable Dupont 15CM F-F 5P 24AWG	1
16	Cable Dupont 8cm F-F 3P 24AWG	1
17	Cable Dupont 4P a 1P Hembra	1
18	Placa Acrílica	1
19	Placa Superior Coche Inteligente 4WD V2.0	1
20	PCB Inferior Coche Inteligente 4WD V2.0	1
21	Piezas Fijas	4
22	Rueda	4
23	Buje de Cobre de Doble Paso M3*10MM	10
24	Buje de Cobre de Doble Paso M3*40MM	4
25	Tornillos de Cabeza Redonda M3*30MM	8
26	Tornillos de Cabeza Redonda M3*6MM	40
27	Tuercas Niqueladas M3	16
28	Tornillos de Cabeza Redonda M2X8MM	6
29	Tornillos de Cabeza Redonda M3*8MM	4
30	Tuercas Niqueladas M2	6

No.	Nombre	Cant.
31	Tornillos Planos M3*10MM	3
32	Motor (con cable de soldadura)	4
33	Destornillador 3*40MM	1

34	Bridas de Nylon Negras 3*100MM	6
35	Tubo de Enrollamiento	1
36	Cable Dupont 3Pin F-F (20CM)	3
37	Placa Decorativa	1

1.5 Placa de Desarrollo Keyestudio V4.0

Es importante saber que la placa de desarrollo Keyestudio V4.0 es el núcleo de este coche inteligente.

La placa Keyestudio V4.0 es compatible con Arduino Uno, se basa en el MCU ATmega328P y cuenta con un chip CP2102 como convertidor UART a USB.

Tiene 14 pines de entrada/salida digital (de los cuales 6 pueden usarse como salidas PWM), 6 entradas analógicas, un cristal de cuarzo de 16 MHz, una conexión USB, un conector de alimentación, 2 encabezados ICSP y un botón de reinicio. Contiene todo lo necesario para soportar el microcontrolador. Simplemente conéctelo a una computadora con un cable USB o alimentelo a través de un conector de alimentación de CC externo (CC 7-12 V) para comenzar.

Especificaciones de la Placa

Microcontrolador	ATmega328P-PU
Voltaje de Operación	5V
Voltaje de Entrada (recomendado)	DC 7-12V
Pines I/O Digitales	14 (D0-D13) (6 con PWM)
Pines I/O Digitales PWM	6 (D3, D5, D6, D9, D10, D11)

Pines de Entrada Analógica	6 (A0-A5)
Corriente DC por Pin I/O	20 mA
Corriente DC para Pin 3.3V	50 mA
Memoria Flash	32 KB (0.5 KB para el cargador de arranque)

SRAM	2 KB
EEPROM	1 KB
Velocidad de Reloj	16 MHz

1.6 Código de Keyestudio V4.0 Arduino

```
//*****
*****
/*
keyestudio 4wd BT Car
lesson 16
Bluetooth Speed Control Car
http://www.keyestudio.com
*/

#define SCL_Pin  A5  //Set the clock pin to A5
#define SDA_Pin  A4  //Set data pin to A4
//Array, used to store the data of pattern, can be calculated by
yourself or obtained from the modulus tool
unsigned char start01[] =
{0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x80,0x40,0x20,0x10,0x08,0x04,
0x02,0x01};
unsigned char front[] =
{0x00,0x00,0x00,0x00,0x00,0x24,0x12,0x09,0x12,0x24,0x00,0x00,0x00,0x00,
0x00,0x00};
unsigned char back[] =
{0x00,0x00,0x00,0x00,0x00,0x24,0x48,0x90,0x48,0x24,0x00,0x00,0x00,0x00,
0x00,0x00};
unsigned char left[] =
{0x00,0x00,0x00,0x00,0x00,0x00,0x44,0x28,0x10,0x44,0x28,0x10,0x44,0x28,
0x10,0x00};
unsigned char right[] =
{0x00,0x10,0x28,0x44,0x10,0x28,0x44,0x10,0x28,0x44,0x00,0x00,0x00,0x00,
0x00,0x00};
```

```

unsigned char STOP01[] =
{0x2E,0x2A,0x3A,0x00,0x02,0x3E,0x02,0x00,0x3E,0x22,0x3E,0x00,0x3E,0x0A,
0x0E,0x00};
unsigned char clear[] =
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
0x00,0x00};
unsigned char speed_a[] =
{0x00,0x40,0x20,0x10,0x08,0x04,0x02,0xff,0x02,0x04,0x08,0x10,0x20,0x40,
0x00,0x00};
unsigned char speed_d[] =
{0x00,0x02,0x04,0x08,0x10,0x20,0x40,0xff,0x40,0x20,0x10,0x08,0x04,0x02,
0x00,0x00};

int left_ctrl = 2;//define the direction control pins of group B motor
int left_pwm = 5;//define the PWM control pins of group B motor
int right_ctrl = 4;//define the direction control pins of group A motor
int right_pwm = 6;//define the PWM control pins of group A motor

int speeds = 150; //Set the initial speed to 150

const int servopin = A3;//set the pin of servo to A3

char BLE_val;

void setup() {
    Serial.begin(9600);//
    pinMode(left_ctrl,OUTPUT);//set direction control pins of group B
motor to OUTPUT
    pinMode(left_pwm,OUTPUT);//set PWM control pins of group B motor to
OUTPUT
    pinMode(right_ctrl,OUTPUT);//set direction control pins of group A
motor to OUTPUT
    pinMode(right_pwm,OUTPUT);//set PWM control pins of group A motor to
OUTPUT
    servopulse(servopin,90);//the angle of servo is 90 degree
    delay(300);
    pinMode(SCL_Pin,OUTPUT);// Set the clock pin to output
    pinMode(SDA_Pin,OUTPUT);//Set the data pin to output
    matrix_display(clear);
    matrix_display(start01); //display start01 expression pattern
}

void loop() {
    if(Serial.available(>0) {
        BLE_val = Serial.read();
    }
}

```

```

    Serial.println(BLE_val);
}
switch(BLE_val)
{
    case 'F' : car_front();
    matrix_display(clear);
    matrix_display(front);
    break;

    case 'B' : car_back();
    matrix_display(clear);
    matrix_display(back);
    break;

    case 'L' : car_left();
    matrix_display(clear);
    matrix_display(left);
    break;

    case 'R' : car_right();
    matrix_display(clear);
    matrix_display(right);
    break;

    case 'S' : car_Stop();
    matrix_display(clear);
    matrix_display(STOP01);
    break;

    case 'a' : speeds_a();
    matrix_display(clear);
    matrix_display(speed_a);
    break;

    case 'd' : speeds_d();
    matrix_display(clear);
    matrix_display(speed_d);
    break;
}
}

void car_front()//define the state of going front
{
    digitalWrite(left_ctrl,HIGH);
    analogWrite(left_pwm,(255-speeds));
}

```



```

    digitalWrite(right_ctrl,HIGH);
    analogWrite(right_pwm,(255-speeds));
}
void car_back()//define the status of going back
{
    digitalWrite(left_ctrl,LOW);
    analogWrite(left_pwm,speeds);
    digitalWrite(right_ctrl,LOW);
    analogWrite(right_pwm,speeds);
}
void car_left()//set the status of left turning
{
    digitalWrite(left_ctrl, LOW);
    analogWrite(left_pwm, speeds);
    digitalWrite(right_ctrl, HIGH);
    analogWrite(right_pwm, (255-speeds));
}
void car_right()//set the status of right turning
{
    digitalWrite(left_ctrl, HIGH);
    analogWrite(left_pwm, (255-speeds));
    digitalWrite(right_ctrl, LOW);
    analogWrite(right_pwm, speeds);
}
void car_Stop()//define the state of stop
{
    digitalWrite(left_ctrl,LOW);
    analogWrite(left_pwm,0);
    digitalWrite(right_ctrl,LOW);
    analogWrite(right_pwm,0);
}

void speeds_a() { //rapidly growing function
    while (1) {
        Serial.println(speeds); //display speed information
        if (speeds < 255) { //Up to 255
            matrix_display(clear);
            matrix_display(speed_a);
            speeds++;
            delay(10); //adjust the speed of growth
        }
        BLE_val = Serial.read();
        if (BLE_val == 'S') //Receive 'S',the car stops accelerating
            break;
    }
}

```

```

}

void speeds_d() { //velocity reduction function
  while (1) {
    Serial.println(speeds); //display speed information
    if (speeds > 0) { //down to 0
      matrix_display(clear);
      matrix_display(speed_d);
      speeds--;
      delay(10); //adjust the speed of deceleration
    }
    BLE_val = Serial.read();
    if (BLE_val == 'S') //Receive 'S',the car stops deceleration
      break;
  }
}

void servopulse(int servopin,int myangle)//Steering gear running angle
{
  for(int i=0; i<30; i++)
  {
    int pulsewidth = (myangle*11)+500;
    digitalWrite(servopin,HIGH);
    delayMicroseconds(pulsewidth);
    digitalWrite(servopin,LOW);
    delay(20-pulsewidth/1000);
  }
}

//this function is used for dot matrix display
void matrix_display(unsigned char matrix_value[])
{
  IIC_start(); //the function that calls the data transfer start
condition
  IIC_send(0xc0); //select address

  for (int i = 0; i < 16; i++) //the pattern data is 16 bytes
  {
    IIC_send(matrix_value[i]); //Transmit the data of the pattern
  }
  IIC_end(); //End pattern data transmission
  IIC_start();
  IIC_send(0x8A); //Display control, select 4/16 pulse width
  IIC_end();
}
//Conditions under which data transmission begins

```

```

void IIC_start()
{
    digitalWrite(SDA_Pin, HIGH);
    digitalWrite(SCL_Pin, HIGH);
    delayMicroseconds(3);
    digitalWrite(SDA_Pin, LOW);
    delayMicroseconds(3);
    digitalWrite(SCL_Pin, LOW);
}
//Indicates the end of data transmission
void IIC_end()
{
    digitalWrite(SCL_Pin, LOW);
    digitalWrite(SDA_Pin, LOW);
    delayMicroseconds(3);
    digitalWrite(SCL_Pin, HIGH);
    delayMicroseconds(3);
    digitalWrite(SDA_Pin, HIGH);
    delayMicroseconds(3);
}
//transmit data
void IIC_send(unsigned char send_data)
{
    for (byte mask = 0x01; mask != 0; mask <= 1) //Each byte has 8 bits
and is checked bit by bit starting at the lowest level
    {
        if (send_data & mask) { //Sets the high and low levels of SDA_Pin
depending on whether each bit of the byte is a 1 or a 0
            digitalWrite(SDA_Pin, HIGH);
        } else {
            digitalWrite(SDA_Pin, LOW);
        }
        delayMicroseconds(3);
        digitalWrite(SCL_Pin, HIGH); //Pull the clock pin SCL_Pin high to
stop data transmission
        delayMicroseconds(3);
        digitalWrite(SCL_Pin, LOW); //pull the clock pin SCL_Pin low to
change the SIGNAL of SDA
    }
}
//*****
*****

```

message.txt

8 KB

```

/*
keyestudio 4wd BT Car V2.0
lesson 12
remote control robot
http://www.keyestudio.com
*/

//Array, used to store the data of pattern, can be calculated by yourself or
obtained from the modulus tool
unsigned char start01[] =
{0x01,0x02,0x04,0x08,0x10,0x20,0x40,0x80,0x80,0x40,0x20,0x10,0x08,0x04,0x02,0x01};
unsigned char front[] =
{0x00,0x00,0x00,0x00,0x00,0x24,0x12,0x09,0x12,0x24,0x00,0x00,0x00,0x00,0x00,0x00};
unsigned char back[] =
{0x00,0x00,0x00,0x00,0x00,0x24,0x48,0x90,0x48,0x24,0x00,0x00,0x00,0x00,0x00,0x00};
unsigned char left[] =
{0x00,0x00,0x00,0x00,0x00,0x00,0x44,0x28,0x10,0x44,0x28,0x10,0x44,0x28,0x10,0x00};
unsigned char right[] =
{0x00,0x10,0x28,0x44,0x10,0x28,0x44,0x10,0x28,0x44,0x00,0x00,0x00,0x00,0x00,0x00};
unsigned char STOP01[] =
{0x2E,0x2A,0x3A,0x00,0x02,0x3E,0x02,0x00,0x3E,0x22,0x3E,0x00,0x3E,0x0A,0x0E,0x00};
unsigned char clear[] =
{0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00};
#define SCL_Pin  A5  //Set clock pin to A5
#define SDA_Pin  A4  //Set data pin to A4
#define ML_Ctrl 4    //define direction control pin of B motor
#define ML_PWM 5     //define PWM control pin of B motor
#define MR_Ctrl 2     //define direction control pin of A motor
#define MR_PWM 6     //define PWM control pin of A motor
#include <IRremote.h> //function library of IR remote control
int RECV_PIN =3; // set the pin of IR receiver to 3
IRrecv irrecv(RECV_PIN);
long irr_val;
decode_results results;
void setup()
{
    pinMode(ML_Ctrl, OUTPUT); //define direction control pin of B motor to OUTPUT
    pinMode(ML_PWM, OUTPUT); //define PWM control pin of B motor to OUTPUT
    pinMode(MR_Ctrl, OUTPUT); //define direction control pin of A motor to OUTPUT
    pinMode(MR_PWM, OUTPUT); //define PWM control pin of A motor to OUTPUT
    Serial.begin(9600); //Start serial printing, baud rate is 9600
    // In case the interrupt driver crashes on setup, give a clue
    // to the user what's going on.
    irrecv.enableIRIn(); // Start the receiver
    Serial.println("Enabled IRin");
    //Set pin to output
    pinMode(SCL_Pin, OUTPUT);
    pinMode(SDA_Pin, OUTPUT);
    //Clear the matrix display

```

```

    matrix_display(clear);
    matrix_display(start01);
}
void loop()
{
    if (irrecv.decode(&results))
    {
        irr_val = results.value;
        Serial.println(irr_val, HEX); //serial reads the IR remote signals
        switch(irr_val)
        {
            case 0xFF629D : car_front(); matrix_display(front); break;
            case 0xFFA857 : car_back(); matrix_display(back); break;
            case 0xFF22DD : car_left(); matrix_display(left); break;
            case 0xFFC23D : car_right(); matrix_display(right); break;
            case 0xFF02FD : car_Stop(); matrix_display(STOP01); break;
        }
        irrecv.resume(); // Receive the next value
    }
}
void car_front() //car goes forward
{
    digitalWrite(ML_Ctrl,HIGH); //set direction control pin of B motor to HIGH
    level
    analogWrite(ML_PWM,200); //Set PWM control speed of B motor to 20
    digitalWrite(MR_Ctrl,HIGH); //set direction control pin of A motor to HIGH
    level
    analogWrite(MR_PWM,200); //Set PWM control speed of A motor to 20
}
void car_back() //car goes back
{
    digitalWrite(ML_Ctrl,LOW); //set direction control pin of B motor to LOW
    analogWrite(ML_PWM,200); //set PWM control speed of B motor to 200
    digitalWrite(MR_Ctrl,LOW); //set direction control pin of A motor to LOW
    analogWrite(MR_PWM,200); //set PWM control speed of A motor to 200
}
void car_left() //car turns left
{
    digitalWrite(ML_Ctrl,LOW); //set direction control pin of B motor to LOW
    analogWrite(ML_PWM,200); //set PWM control speed of B motor to 200
    digitalWrite(MR_Ctrl,HIGH); //set direction control pin of A motor to HIGH
    level
    analogWrite(MR_PWM,200); //set PWM control speed of A motor to 200
}
void car_right() //car turns right
{
    digitalWrite(ML_Ctrl,HIGH); //set direction control pin of B motor to HIGH
    level
    analogWrite(ML_PWM,200); //set PWM control speed of B motor to 200
    digitalWrite(MR_Ctrl,LOW); //set direction control pin of A motor to LOW
    analogWrite(MR_PWM,200); //set PWM control speed of A motor to 200
}
}

```

```

void car_Stop()//car stops
{
    analogWrite(ML_PWM,0);//set PWM control speed of B motor to 0
    analogWrite(MR_PWM,0);//set PWM control speed of A motor to 0
}
//this function is used for dot matrix display
void matrix_display(unsigned char matrix_value[])
{
    IIC_start(); //the function to call the data transmission
    IIC_send(0xc0); //Select address
    for(int i = 0;i < 16;i++) //Pattern data has 16 bytes
    {
        IIC_send(matrix_value[i]); //data to convey patterns
    }
    IIC_end(); //end the transmission of patterns data
    IIC_start();
    IIC_send(0x8A); //display control, set pulse width to 4/16
    IIC_end();
}
// the condition that data transmission starts
void IIC_start()
{
    digitalWrite(SCL_Pin,HIGH);
    delayMicroseconds(3);
    digitalWrite(SDA_Pin,HIGH);
    delayMicroseconds(3);
    digitalWrite(SDA_Pin,LOW);
    delayMicroseconds(3);
}
// transmit data
void IIC_send(unsigned char send_data)
{
    for(char i = 0;i < 8;i++) //Every character has 8 bits
    {
        digitalWrite(SCL_Pin,LOW); //pull down the SCL_Pin to change the signal
of SDA
        delayMicroseconds(3);
        if(send_data & 0x01) //1 or 0 of byte is used to set high and low level
of SDA_Pin
        {
            digitalWrite(SDA_Pin,HIGH);
        }
        else
        {
            digitalWrite(SDA_Pin,LOW);
        }
        delayMicroseconds(3);
        digitalWrite(SCL_Pin,HIGH); //Pull up SCL_Pin to stop data transmission
        delayMicroseconds(3);
        send_data = send_data >> 1; //Detect bit by bit, so move the data right
by one bit
    }
}

```

```
}  
//the sign that data transmission ends  
void IIC_end()  
{  
    digitalWrite(SCL_Pin, LOW);  
    delayMicroseconds(3);  
    digitalWrite(SDA_Pin, LOW);  
    delayMicroseconds(3);  
    digitalWrite(SCL_Pin, HIGH);  
    delayMicroseconds(3);  
    digitalWrite(SDA_Pin, HIGH);  
    delayMicroseconds(3);  
}ooo
```